

Machine Learning Operations (AIMLCZG523)
Assignment 1

CardioRisk

End-to-end MLOps pipeline for heart-disease risk prediction:
experiment tracking, CI/CD, containerisation, Kubernetes deployment and
monitoring

Submission details	
Student name	Gaurisha Mathur
BITS ID	2025ae05642
Code repository	https://github.com/2025ae05642/heart-disease-mlops
Dataset	UCI Heart Disease (Cleveland subset, 303 patients, 13 features)
Date of submission	October 2026

1. Introduction and objectives

Heart disease is the leading cause of death worldwide, and an early, cheap risk estimate from routine measurements helps clinicians decide who needs further testing. The goal of this assignment is not only a good classifier but a **production-shaped system around it**: reproducible data preparation, tracked experiments, automated testing, a containerised API, a Kubernetes deployment and live monitoring.

The final system, **CardioRisk**, takes 13 clinical measurements as JSON and returns a binary prediction, the probability of disease, a confidence score and a risk band. Every stage from the raw UCI file to the running pods is automated and verified in a GitHub Actions pipeline.

Assignment task	Where it is addressed	Key evidence
1. Data acquisition & EDA	Section 4	download script, 6 EDA figures
2. Feature engineering & models	Section 5	3 model families, 5-fold CV, 5 metrics
3. Experiment tracking	Section 6	MLflow runs, nested grid runs, registry
4. Packaging & reproducibility	Section 7	sklearn Pipeline, joblib + MLflow model, pinned deps
5. CI/CD & testing	Section 8	26 pytest tests, 5-job GitHub Actions workflow
6. Containerisation	Section 9	Dockerfile, /predict smoke test
7. Production deployment	Section 10	Minikube + kind, LoadBalancer, Ingress, HPA
8. Monitoring & logging	Section 11	JSON logs, Prometheus, Grafana dashboard
9. Documentation	This report, README	architecture diagram, screenshots

2. System architecture

The system is organised in three layers (Figure 1). **Experimentation** turns the raw UCI file into a validated dataset, explores it and trains candidate models, with every run recorded in MLflow. **CI/CD** re-executes the whole chain on every push: lint, tests, training with a quality gate, image build with a container smoke test, and a throw-away Kubernetes deployment that is verified end to end. **Serving** runs the image as a replicated Deployment behind a LoadBalancer Service and an NGINX Ingress, with Prometheus discovering every pod and Grafana visualising service and model health.

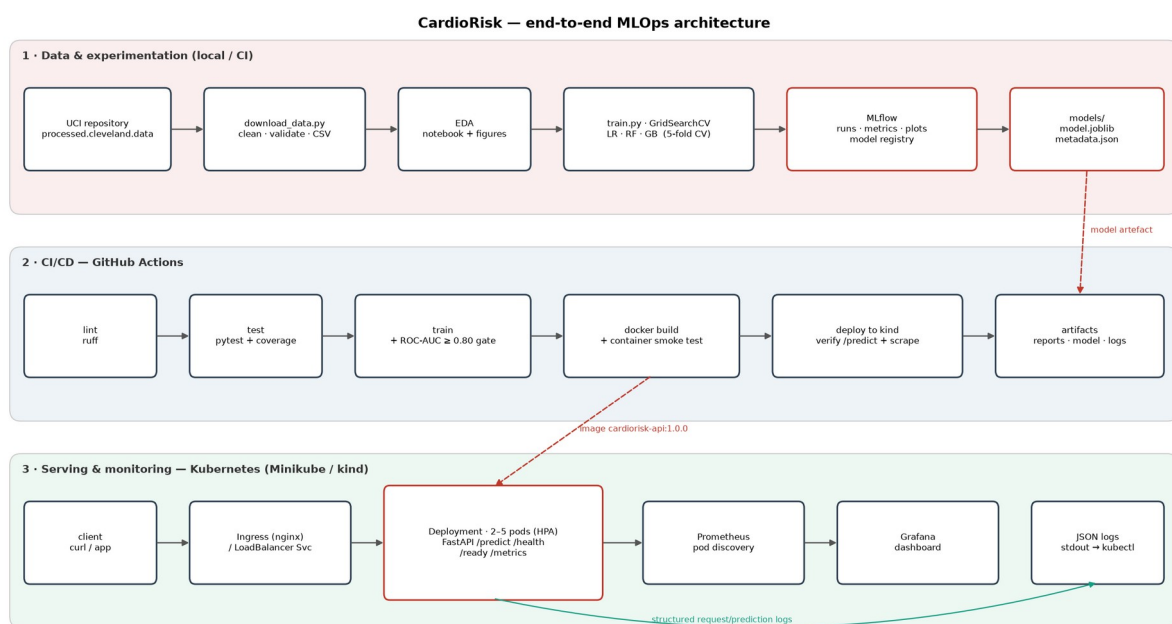


Figure 1. End-to-end architecture of CardioRisk.

Key design decisions

- **One sklearn Pipeline from raw features to prediction.** Feature engineering, imputation, scaling and encoding live inside the saved artefact, so training, notebooks, tests and the API can never apply different transformations (no training/serving skew).
- **The model is baked into the image.** An image tag therefore identifies exactly one model version, and rollbacks are a plain `kubectl rollout undo`.
- **Runtime-only serving image.** The container installs `requirements-serve.txt` (no MLflow, Jupyter or plotting libraries), which keeps it small and reduces the attack surface. Versions of numpy, pandas and scikit-learn are pinned identically in both files so the pickled pipeline always loads.
- **Scale with replicas, not workers.** Prometheus counters live in process memory, so each container runs a single Uvicorn worker and Kubernetes scales pods (HPA 2–5). Prometheus scrapes each pod individually through Kubernetes pod discovery instead of scraping the Service, which would sample a random pod.

3. Setup and installation

Requirements: Python 3.11+ (3.12 is used in CI and in the image), Docker, and for deployment `kubectl` with Minikube or Docker Desktop Kubernetes. All commands run from a clean clone:

```
git clone https://github.com/2025ae05642/heart-disease-mlops.git && cd heart-disease-mlops
python -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt && pip install -e . --no-deps
python scripts/download_data.py      # raw + cleaned data
python scripts/run_eda.py             # EDA figures -> reports/figures/
python -m cardiorisk.train            # tune, compare, track, package
pytest -v && ruff check .             # tests + lint
mlflow ui --backend-store-uri sqlite:///mlflow.db # browse experiments
```

The repository layout follows the deliverables list: `src/cardiorisk/` (package), `scripts/`, `notebooks/` (EDA, training, inference — committed with outputs), `tests/`, `data/raw` and `data/processed`, `models/`, `deploy/k8s/` and `deploy/monitoring/` (manifests), `.github/workflows/ci.yml`, `screenshots/` and `docs/`.

4. Data acquisition and exploratory analysis

4.1 Acquisition and cleaning

`scripts/download_data.py` fetches `processed.cleveland.data` from the UCI repository and falls back to the zipped bundle on UCI's newer static host if the classic URL fails. The file's SHA-256 fingerprint is logged with every training run, so each model can be traced back to the exact bytes it was trained on. Cleaning (`cardiorisk/data.py`) performs four steps:

1. Parse `?` markers as missing values (4 rows in `ca`, 2 rows in `thal`).
2. Collapse the diagnosis `num` (0–4) into a binary target: 0 = no disease, 1–4 = disease.
3. Validate physiological ranges and category codes; impossible values become missing rather than being silently kept.
4. Drop exact duplicate rows (none in this file, but the check protects future data refreshes).

Missing values are deliberately **not** imputed during cleaning. Imputation is part of the model pipeline, so its statistics are learned on training folds only and cross-validation scores are not inflated by leakage.

4.2 Findings

The cleaned dataset has 303 patients: 164 without disease (54 %) and 139 with disease (46 %). The classes are close to balanced, so accuracy is meaningful, but precision, recall and ROC-AUC are reported alongside it because a missed diagnosis (false negative) costs more than a false alarm.

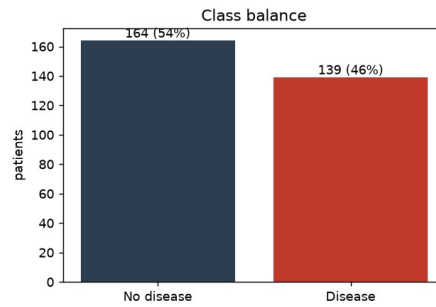


Figure 2. Class balance of the binary target.

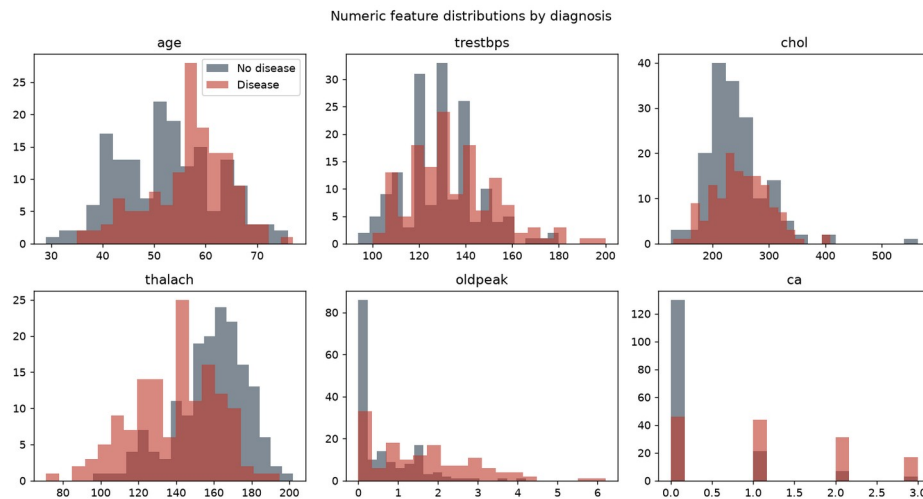


Figure 3. Distributions of numeric features by diagnosis.

Patients with disease reach a lower maximum heart rate (**thalach**), show more exercise-induced ST depression (**oldpeak**) and have more vessels coloured by fluoroscopy (**ca**). Cholesterol and resting blood pressure overlap heavily between classes.

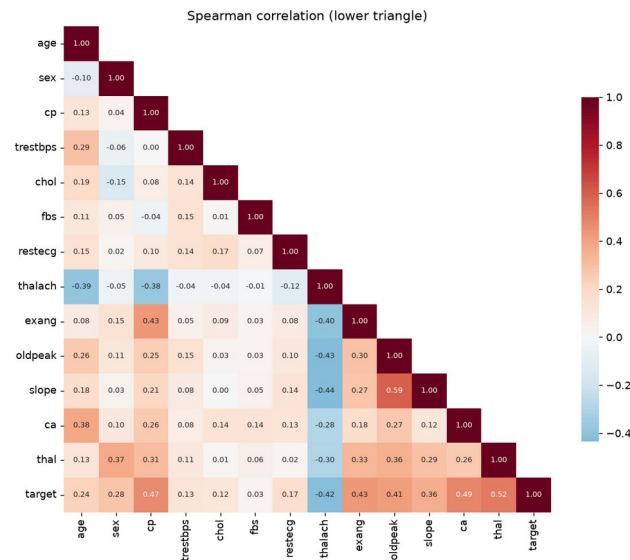


Figure 4. Spearman correlation matrix (rank-based, robust to the ordinal codes).

Spearman correlation with the target ranks the signals as: **thal** 0.52, **ca** 0.49, **cp** 0.47, **exang** 0.43, **thalach** -0.42, **oldpeak** 0.41, **slope** 0.36, then much weaker **sex**, **age**, **restecg**, **trestbps**, **chol** and **fbs** (0.03).

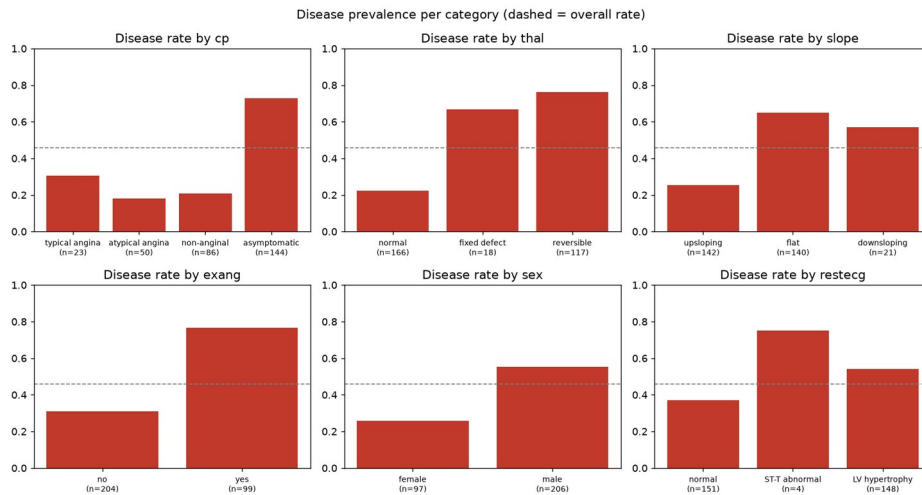


Figure 5. Disease prevalence per category; the dashed line is the overall rate.

Asymptomatic chest pain ($cp = 4$), a reversible thallium defect and a flat ST slope roughly double the disease rate relative to baseline. These are **nominal codes**: the numbers 1–4 carry no order, so they must be one-hot encoded rather than treated as numeric.

4.3 Decisions driven by EDA

Observation	Decision
<code>cp</code> , <code>thal</code> , <code>slope</code> , <code>restecg</code> are nominal codes	One-hot encode with a fixed category list (stable width even if a CV fold misses a level)
Max heart rate falls with age, confounding <code>thalach</code>	Engineer <code>hr_reserve_pct</code> = <code>thalach</code> / (220 – age)
Cholesterol and BP are individually weak	Engineer interaction <code>bp_chol_load</code> = <code>z</code> (BP) × <code>z</code> (chol)
Feature scales differ by two orders of magnitude	StandardScaler on numeric features (needed for the linear model)
Only 6 missing cells, all in categorical-like fields	Median / most-frequent imputation inside the pipeline
Outliers in <code>chol</code> / <code>trestbps</code> are physiologically plausible	Kept; tree models are robust and scaling limits their effect on LR

5. Feature engineering and model development

5.1 Pipeline

The model is a single sklearn Pipeline: `ClinicalFeatureBuilder` → `ColumnTransformer` → estimator. The custom transformer adds the two engineered features and learns the BP and cholesterol reference statistics in `fit()`, so they come from training data only. The column transformer applies median imputation and scaling to the 8 numeric features (6 raw + 2 engineered), most-frequent imputation to the 3 binary flags, and imputation plus one-hot encoding (13 columns) to the 4 nominal features, giving 24 model inputs.

5.2 Candidate models and tuning

Three model families were compared, each tuned with `GridSearchCV` over a deliberately small grid. With about 240 training rows, wide grids mostly fit noise in the CV estimate.

Model	Search space	Candidates	Selected hyper-parameters
Logistic Regression	$C \in \{0.01, 0.1, 0.3, 1, 3\}$; L1 vs L2 (<code>l1_ratio</code> ∈ {0, 1}); <code>class_weight</code> ∈ {None, balanced}	20	$C = 1.0$, L2, balanced
Random Forest	<code>n_estimators</code> ∈ {200, 400}; <code>max_depth</code> ∈ {4, 6, None}; <code>min_samples_leaf</code> ∈ {2, 5}; <code>max_features</code> ∈ {sqrt, 0.5}	24	400 trees, depth 6, leaf 5, sqrt

Model	Search space	Candidates	Selected hyper-parameters
Gradient Boosting	n_estimators ∈ {100, 200}; learning_rate ∈ {0.03, 0.1}; max_depth ∈ {2, 3}; subsample 0.8	8	100 trees, lr 0.03, depth 2

Protocol. A stratified 80/20 split (seed 42) sets aside 61 patients as a hold-out test set. Model selection uses stratified 5-fold cross-validation on the 242 training patients only, scoring accuracy, precision, recall, F1 and ROC-AUC. Each grid is refit on ROC-AUC because it is threshold-independent and measures how well the model ranks patients by risk. **The winner is chosen by mean CV ROC-AUC; the hold-out set is used only for reporting**, so its numbers remain an unbiased estimate.

5.3 Results

Model	CV ROC-AUC	CV accuracy	CV recall	Test ROC-AUC	Test accuracy	Test precision	Test recall
Logistic Regression	0.907 ± 0.015	0.831	0.793	0.958	0.869	0.813	0.929
Random Forest	0.896 ± 0.025	0.814	0.765	0.954	0.885	0.839	0.929
Gradient Boosting	0.881 ± 0.021	0.818	0.765	0.957	0.902	0.867	0.929

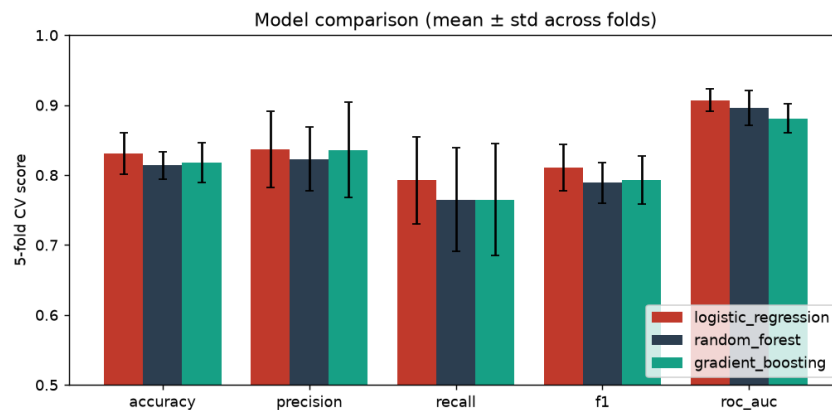


Figure 6. Five-fold CV comparison (error bars = standard deviation across folds).

Why Logistic Regression. It has the highest mean CV ROC-AUC and the lowest fold-to-fold variance. The tree ensembles score slightly higher accuracy on the 61-patient hold-out, but a difference of one or two patients is within noise at that size, and the CV estimate (five times as many evaluations) favours LR. LR is also the easiest to explain to clinicians: its largest coefficients are male sex, number of coloured vessels, asymptomatic chest pain and reversible thallium defect (risk ↑), and normal thallium, typical angina and upsloping ST (risk ↓). This matches the clinical literature and the EDA. The balanced class weight also gives the model the best recall on the hold-out set (26 of 28 diseased patients detected; 6 false alarms among 33 healthy patients).

6. Experiment tracking with MLflow

Every training run logs to MLflow (SQLite backend locally; the database is uploaded as a CI artefact). Runs are nested so a single training produces 56 runs: one **parent** selection run, one **family** run per model type, and one **candidate** run per grid point.

Run level	Parameters	Metrics	Artefacts
Parent (model-selection-<ts>)	row counts, CV folds, seed, data SHA-256, positive rate	winner CV / test ROC-AUC	cleaned CSV, comparison chart + CSV, metadata.json, registered final model

Run level	Parameters	Metrics	Artefacts
Family (e.g. logistic_regression)	best hyper-parameters	CV mean & std of 5 metrics, hold-out metrics	ROC, PR, confusion matrix, feature importance (PNG + CSV), cv_results.csv, MLflow model with signature + input example
Candidate (...-cand-NN)	that grid point	CV mean & std of 5 metrics, rank	—

The final model is registered as **cardiorisk-classifier** in the MLflow Model Registry; two versions exist, one from the CLI run and one from the training notebook. MLflow 3.x serialises sklearn models with skops by default, which refuses unknown classes. Instead of falling back to pickle, the code passes an explicit allow-list of trusted types: the project's own transformer and sklearn's tree structure.

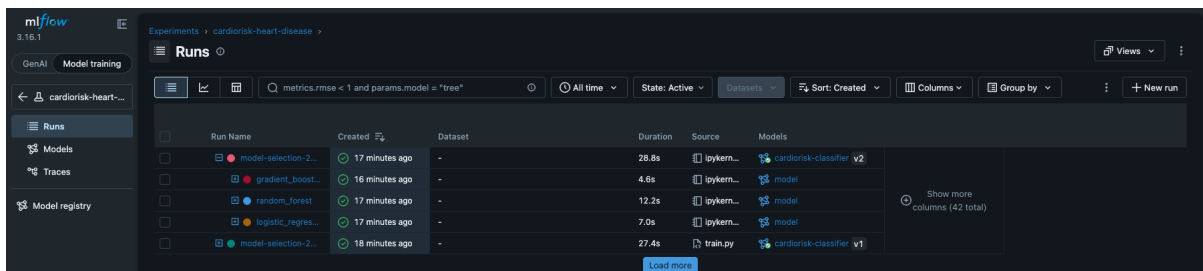


Figure 7. MLflow runs: parent selection runs with nested family runs and registered model versions.

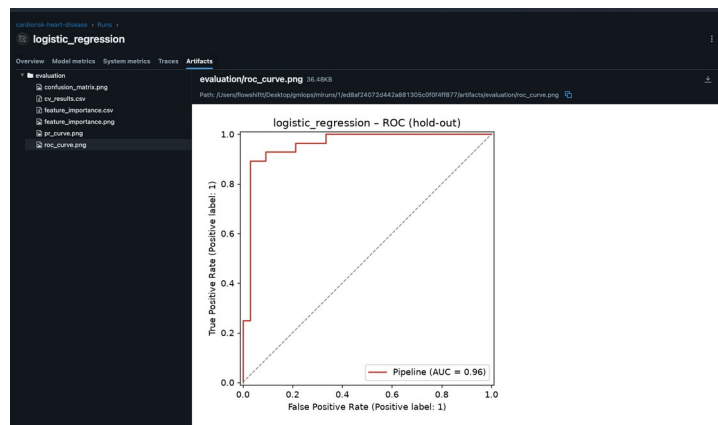


Figure 8. Artefacts logged for the winning run: evaluation plots and CSVs.

7. Model packaging and reproducibility

- **Artefacts.** **models/model.joblib** (the full pipeline, loaded by the API), **models/mlflow_model/** (MLflow sklearn/pyfunc flavour with signature and input example), and **models/metadata.json** (model name, version, training timestamp, MLflow run id, data hash, features, threshold, best parameters, CV and test metrics, Python and sklearn versions).
- **Dependencies.** **requirements.txt** (pinned runtime + training), **requirements-dev.txt** (adds pytest, ruff, nbconvert) and **requirements-serve.txt** (the container's minimal subset, same pins for the numeric stack).
- **Determinism.** Fixed seeds for the split, CV folds and every estimator. Re-running training reproduces identical metrics. The notebook re-run produced exactly the same hold-out ROC-AUC of 0.9578 as the CLI run.
- **Inference parity.** **notebooks/03_inference.ipynb** loads both the joblib pipeline and the MLflow pyfunc model and scores the same patients, including one with unknown **ca/thal**, which the pipeline imputes.

8. CI/CD pipeline and automated testing

8.1 Unit tests

26 pytest tests run offline in about 2 seconds on synthetic data with a planted signal, so CI does not depend on the UCI server.

File	What is verified
test_data.py (7)	binary target, NaNs preserved for imputation, out-of-range and invalid codes nulled, duplicates dropped, '?' parsing, X/y shapes and dtypes
test_features.py (5)	heart-rate-reserve formula, no leakage of transform-time statistics, no NaNs after preprocessing and correct width, unknown categories tolerated, standardisation
test_model.py (7)	both LR and RF learn the signal (CV AUC > 0.7), valid probabilities, metric dictionary, importances aligned with names, required model families present, save/load round-trip, missing optional fields
test_api.py (7)	health/readiness, /predict returns prediction + confidence, 422 on out-of-range and unknown fields, batch endpoint, Prometheus metrics exposed, JSON request logs carry the request id

8.2 GitHub Actions workflow

`.github/workflows/ci.yml` runs on every push and pull request (and on demand, with an optional quick-grid mode). Jobs are chained with `needs:`, and every step runs under `bash -euo pipefail`. Any failing command therefore stops the pipeline with a clear log and skips everything downstream.

Job	Steps	Artefacts / logging
1. Lint	ruff check (pyflakes, pycodestyle, isort, bugbear, pyupgrade)	inline GitHub annotations
2. Test	pip install pinned deps, record pip freeze, pytest + coverage	JUnit XML, coverage XML, pip-freeze.txt
3. Train	download data, EDA, full training, quality gate: hold-out ROC-AUC ≥ 0.80	model, MLflow DB + runs, figures; metrics in job summary
4. Docker	build image, run container, smoke test, save image	container logs, image tarball
5. Deploy	kind cluster, load image, apply manifests + monitoring, rollout, smoke test via Service, assert Prometheus sees ≥ 2 healthy replicas	cluster state (get/describe/logs)

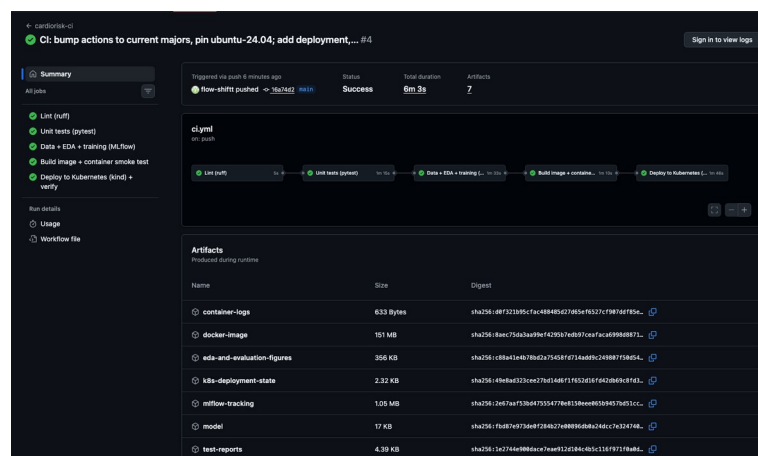


Figure 9. Successful pipeline run: all five jobs green, with the seven uploaded artefacts (model, MLflow store, figures, test reports, image, logs, cluster state).

The very first push failed at the lint stage, and test, train, build and deploy were all skipped (Figure 10). This is exactly the fail-fast behaviour the assignment asks for. The run history also shows a deploy-job failure that was

traced to curl treating `{}` in a Prometheus query as a URL glob; it was fixed by URL-encoding the query and adding a retry loop.

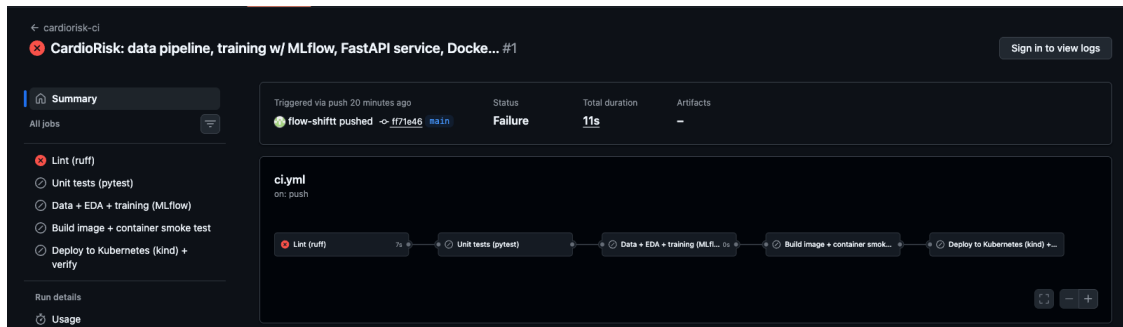


Figure 10. Pipeline stops at the first failing stage (lint); downstream jobs are skipped.

9. Model containerisation

The image is built from `python:3.12-slim` and installs only the serving dependencies. It copies the package and the model artefact, runs as a non-root user (UID 10001), and declares a Docker `HEALTHCHECK` against `/health`. The API is FastAPI with Pydantic validation: each field has a physiological range or an allowed set of codes, unknown fields are rejected, and `ca/thal` may be null (imputed by the pipeline).

Endpoint	Purpose
POST /predict	One patient → prediction (0/1), label, confidence (probability of the predicted class), probability_disease, risk_band, model_name, model_version, request_id
POST /predict/batch	Up to 500 patients in one call
GET /health, /ready	Liveness (process up) and readiness (model loaded), used by Kubernetes probes
GET /model-info	Contents of metadata.json for the loaded model
GET /metrics	Prometheus exposition format

```
docker build -t cardiorisk-api:1.0.0 .
docker run -d -p 8000:8000 --name cardiorisk-api cardiorisk-api:1.0.0
curl -X POST localhost:8000/predict -H 'content-type: application/json' -d @sample_request.json
{"prediction":1,"label":"disease","confidence":0.9969,"probability_disease":0.9969,
"risk_band":"high","model_name":"logistic_regression","model_version":"1.0.0+202610030813",...}
```

```
docker run cardiorisk-api:1.0.0 - smoke test

$ docker ps
CONTAINER ID   IMAGE               COMMAND                  CREATED        STATUS        PORTS
305a7fc2b05a   cardiorisk-api:1.0.0   "uvicorn cardiorisk..."   Less than a second ago   Up Less than a second (health: starting)
0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp   cardiorisk-api

$ bash scripts/smoke_test.sh http://localhost:8000
== waiting for http://localhost:8000/ready
== GET /health
{"status":"ok"}
== POST /predict (high-risk sample)
{"prediction":1,"label":"disease","confidence":0.9969,"probability_disease":0.9969,"risk_band":"high","model_name":"logistic_regression","model_version":"1.0.0+202610030813","request_id":"fee4d8a495c3"}
== POST /predict (invalid payload must be rejected with 422)
422 OK
== GET /metrics (excerpt)
cardiorisk_predictions_total{label="disease",risk_band="high"} 1.0
cardiorisk_model_info{name="logistic_regression",version="1.0.0+202610030813"} 1.0
SMOKE TEST PASSED

$ docker logs cardiorisk-api
INFO: Application startup complete.
{"ts":"2026-10-03T08:21:57.074+00:00","level":"INFO","logger":"cardiorisk.api","msg":"model_loaded","model_name":"logistic_regression","model_version":"1.0.0+202610030813"}
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
{"ts":"2026-10-03T08:21:58.006+00:00","level":"INFO","logger":"cardiorisk.api","msg":"request","request_id":"82842923bb24","method":"GET","route":"/ready","status":200,"latency_ms":1.62,"client":"172.17.0.1"}
{"ts":"2026-10-03T08:21:58.019+00:00","level":"INFO","logger":"cardiorisk.api","msg":"request","request_id":"105810dbaf65","method":"GET","route":"/health","status":200,"latency_ms":0.44,"client":"172.17.0.1"}
{"ts":"2026-10-03T08:21:58.042+00:00","level":"INFO","logger":"cardiorisk.api","msg":"prediction","request_id":"fee4d8a495c3","label":"disease","probability_disease":0.9969,"risk_band":"high","model_version":"1.0.0+202610030813"}
{"ts":"2026-10-03T08:21:58.042+00:00","level":"INFO","logger":"cardiorisk.api","msg":"request","request_id":"fee4d8a495c3","method":"POST","route":"/predict","status":200,"latency_ms":13.91,"client":"172.17.0.1"}
{"ts":"2026-10-03T08:21:58.057+00:00","level":"INFO","logger":"cardiorisk.api","msg":"request","request_id":"bdf599a35c46","method":"POST","route":"/predict","status":422,"latency_ms":0.41,"client":"172.17.0.1"}
```

Figure 11. Local container: build, run and automated smoke test (health, prediction, 422 on invalid input, metrics) with structured logs.

10. Production deployment on Kubernetes

The image was deployed to a local **Minikube** cluster (Docker driver on Colima). CI also deploys it to an ephemeral **kind** cluster on every push. Manifests in `deploy/k8s/` are applied together with `kubectl apply -k:`

- **Deployment:** 2 replicas, rolling update with `maxUnavailable: 0` (no capacity drop during a model rollout), readiness probe on `/ready` and liveness probe on `/health`, CPU/memory requests and limits, and a hardened security context (non-root, read-only root filesystem, all capabilities dropped, seccomp RuntimeDefault).
- **Service** of type **LoadBalancer** (port 80 → 8000). Minikube assigns it an external IP via `minikube tunnel`; on GKE, EKS or AKS the same manifest provisions a cloud load balancer.
- **Ingress** (NGINX class) routing host `cardiorisk.local` to the Service.
- **HorizontalPodAutoscaler:** 2-5 replicas at 70 % CPU.

```
minikube - kubectl get (namespace cardiorisk)

$ kubectl -n cardiorisk get deploy,pods,svc,ingress,hpa
NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
deployment.apps/cardiorisk-api       2/2      2              2            5m38s
deployment.apps/grafana              1/1      1              1            4m15s
deployment.apps/prometheus           1/1      1              1            4m15s

NAME                                READY    STATUS    RESTARTS    AGE
pod/cardiorisk-api-5466cf7f87-kfx4s  1/1      Running   0            5m38s
pod/cardiorisk-api-5466cf7f87-wr29m  1/1      Running   0            5m38s
pod/grafana-589567dc75-z2kjq         1/1      Running   0            56s
pod/prometheus-7585bd9d85-pt2zd      1/1      Running   0            4m15s

NAME                                TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
service/cardiorisk-api              LoadBalancer  10.104.59.12  127.0.0.1      80:31428/TCP     5m38s
service/grafana                     ClusterIP      10.99.132.145 <none>         3000/TCP         4m16s
service/prometheus                  ClusterIP      10.100.172.83 <none>         9090/TCP         4m16s

NAME                                CLASS    HOSTS          ADDRESS        PORTS    AGE
ingress.networking.k8s.io/cardiorisk-api  nginx    cardiorisk.local  192.168.49.2  80       4m23s

NAME                                REFERENCE    TARGETS    MINPODS    MAXPODS    REPLICAS    AGE
horizontalpodautoscaler.autoscaling/cardiorisk-api  Deployment/cardiorisk-api  cpu: 32%/70%  2          5          2            5m38s
```

Figure 12. All resources in the `cardiorisk` namespace: API pods, monitoring, LoadBalancer service with external IP, ingress and HPA.

```
minikube - ingress (nginx) routing

$ kubectl -n cardiorisk get ingress cardiorisk-api
NAME                                CLASS    HOSTS          ADDRESS        PORTS    AGE
cardiorisk-api                      nginx    cardiorisk.local  192.168.49.2  80       52s

$ kubectl -n ingress-nginx port-forward svc/ingress-nginx-controller 8081:80 &
$ curl -s -H 'Host: cardiorisk.local' http://127.0.0.1:8081/health
{"status":"ok"}
$ curl -s -H 'Host: cardiorisk.local' -X POST http://127.0.0.1:8081/predict -H 'content-type: application/json' -d @sample_request.json
{"prediction":1,"label":"disease","confidence":0.9969,"probability_disease":0.9969,"risk_band":"high","model_name":"logistic_regression","model_version":"1.0.0+202610030813","request_id":"c98b3e8bd8d94df2b3961d1896101e10"}
```

Figure 13. API reached through the NGINX Ingress (Host: `cardiorisk.local`). The full LoadBalancer smoke-test log is in `docs/logs/k8s_deploy_verify.log`.

Access instructions (local testing)

```
minikube start --driver=docker && minikube addons enable ingress
minikube image load cardiorisk-api:1.0.0
kubectl apply -k deploy/k8s && kubectl apply -k deploy/monitoring
minikube tunnel # LoadBalancer gets EXTERNAL-IP (asks for sudo for port 80)
curl http://<EXTERNAL-IP>/health
# without sudo: minikube service cardiorisk-api -n cardiorisk --url
```

11. Monitoring and logging

Logging. A middleware assigns every request an id (or reuses an incoming `x-request-id`, which the NGINX ingress supplies), times it, and writes one JSON object per line to stdout. The object holds the timestamp, route, status, latency and client; prediction events additionally carry the label, probability, risk band and model version. Raw patient features are deliberately not logged. JSON lines can be read with `kubectl logs` and are ready for a log stack such as Loki or ELK.

```

structured JSON request logs from both replicas

$ kubectl -n cardiork logs -l app.kubernetes.io/name=cardiork-api --tail=6 --prefix
[pod/cardiork-api-5466cf7f87-kfx4s/api] {"ts": "2026-10-03T08:28:29.537+00:00", "level": "INFO", "logger": "cardiork.api", "msg": "request",
"request_id": "af5a64aabc83", "method": "GET", "route": "/ready", "status": 200, "latency_ms": 0.95, "client": "10.244.0.1"}
[pod/cardiork-api-5466cf7f87-kfx4s/api] {"ts": "2026-10-03T08:28:34.541+00:00", "level": "INFO", "logger": "cardiork.api", "msg": "request",
"request_id": "04b99ddb1d2b", "method": "GET", "route": "/ready", "status": 200, "latency_ms": 1.42, "client": "10.244.0.1"}
[pod/cardiork-api-5466cf7f87-kfx4s/api] {"ts": "2026-10-03T08:28:39.547+00:00", "level": "INFO", "logger": "cardiork.api", "msg": "request",
"request_id": "c2ee8c181f93", "method": "GET", "route": "/ready", "status": 200, "latency_ms": 4.23, "client": "10.244.0.1"}
[pod/cardiork-api-5466cf7f87-kfx4s/api] {"ts": "2026-10-03T08:28:44.218+00:00", "level": "INFO", "logger": "cardiork.api", "msg": "request",
"request_id": "ca531626c50", "method": "GET", "route": "/health", "status": 200, "latency_ms": 1.05, "client": "10.244.0.1"}
[pod/cardiork-api-5466cf7f87-kfx4s/api] {"ts": "2026-10-03T08:28:44.537+00:00", "level": "INFO", "logger": "cardiork.api", "msg": "request",
"request_id": "079bc39ce992", "method": "GET", "route": "/ready", "status": 200, "latency_ms": 0.40, "client": "10.244.0.1"}
[pod/cardiork-api-5466cf7f87-kfx4s/api] {"ts": "2026-10-03T08:28:49.548+00:00", "level": "INFO", "logger": "cardiork.api", "msg": "request",
"request_id": "7ae3c138b33a", "method": "GET", "route": "/ready", "status": 200, "latency_ms": 3.85, "client": "10.244.0.1"}
[pod/cardiork-api-5466cf7f87-wr29m/api] {"ts": "2026-10-03T08:28:30.211+00:00", "level": "INFO", "logger": "cardiork.api", "msg": "request",
"request_id": "cb1da3ee387b", "method": "GET", "route": "/ready", "status": 200, "latency_ms": 0.6, "client": "10.244.0.1"}
[pod/cardiork-api-5466cf7f87-wr29m/api] {"ts": "2026-10-03T08:28:35.214+00:00", "level": "INFO", "logger": "cardiork.api", "msg": "request",
"request_id": "2e31b1ba6165", "method": "GET", "route": "/ready", "status": 200, "latency_ms": 2.11, "client": "10.244.0.1"}
[pod/cardiork-api-5466cf7f87-wr29m/api] {"ts": "2026-10-03T08:28:40.215+00:00", "level": "INFO", "logger": "cardiork.api", "msg": "request",
"request_id": "29ec57beb607", "method": "GET", "route": "/ready", "status": 200, "latency_ms": 2.23, "client": "10.244.0.1"}
[pod/cardiork-api-5466cf7f87-wr29m/api] {"ts": "2026-10-03T08:28:44.235+00:00", "level": "INFO", "logger": "cardiork.api", "msg": "request",
"request_id": "1752769b1de1", "method": "GET", "route": "/health", "status": 200, "latency_ms": 1.16, "client": "10.244.0.1"}
[pod/cardiork-api-5466cf7f87-wr29m/api] {"ts": "2026-10-03T08:28:45.213+00:00", "level": "INFO", "logger": "cardiork.api", "msg": "request",
"request_id": "cb93fa21ed75", "method": "GET", "route": "/ready", "status": 200, "latency_ms": 1.42, "client": "10.244.0.1"}
[pod/cardiork-api-5466cf7f87-wr29m/api] {"ts": "2026-10-03T08:28:50.214+00:00", "level": "INFO", "logger": "cardiork.api", "msg": "request",
"request_id": "4939b583ce93", "method": "GET", "route": "/ready", "status": 200, "latency_ms": 1.82, "client": "10.244.0.1"}

```

Figure 14. Structured request and prediction logs from both replicas (kubectl logs --prefix).

Metrics. The API exposes the following on `/metrics`:

- request count by route and status, and request latency histograms;
- predictions by label and risk band;
- a histogram of predicted probabilities, used as a simple input-drift signal;
- validation errors by offending field;
- a model-info gauge carrying the loaded model's name and version.

In the cluster, Prometheus uses Kubernetes pod discovery (namespaced RBAC) to scrape every API replica, and Grafana is provisioned automatically with a datasource and an 11-panel dashboard. To produce realistic traffic, `scripts/load_test.py` replays real patients from the dataset, about 5 % of them deliberately invalid.

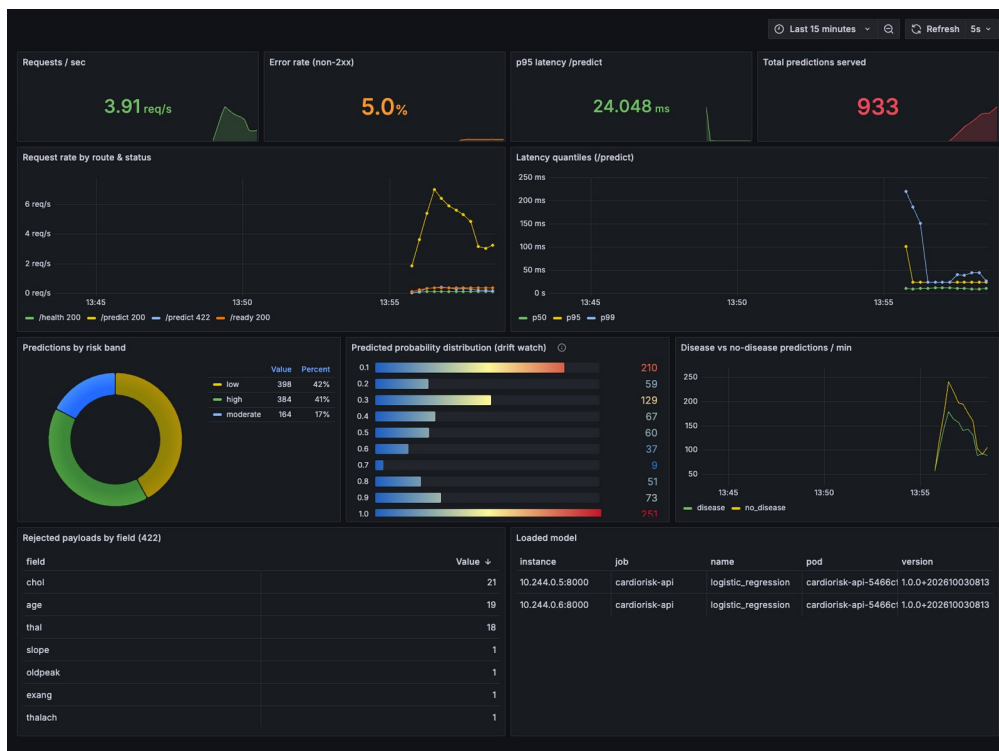


Figure 15. Grafana dashboard under replayed traffic: throughput, error rate, latency quantiles, prediction mix, probability distribution, rejected fields and loaded model per pod.

The dashboard showed p95 latency of about 24 ms for `/predict`, an error rate of about 5 %, consistent with the deliberately injected invalid payloads, and the most frequently rejected fields (`chol`, `age`, `thal`), which tells a client team which inputs they get wrong most often.

12. Limitations and next steps

- **Small dataset.** 303 patients from one hospital; hold-out metrics carry wide confidence intervals (± 1 patient $\approx \pm 1.6$ % accuracy). External validation on the Hungarian, Swiss and VA subsets is the next step before any clinical claim.
- **Threshold.** A fixed 0.5 cut-off. A screening use case would lower it to trade precision for recall, chosen on CV predictions.
- **Drift.** Drift is only watched (probability histogram). Alerting rules and a scheduled retraining job are not yet implemented.
- **Registry-driven deploys.** Images are currently tagged by version. Promoting models via MLflow registry aliases and pushing images to a registry (GHCR) would complete continuous delivery to a cloud cluster.

13. Deliverables and links

Deliverable	Location
Code repository	https://github.com/2025ae05642/heart-disease-mlops
Dockerfile, requirements files	Dockerfile, requirements.txt, requirements-dev.txt, requirements-serve.txt
Cleaned dataset + download script	data/processed/heart_clean.csv, data/raw/, scripts/download_data.py
Notebooks	notebooks/01_eda.ipynb, 02_training.ipynb, 03_inference.ipynb
Unit tests	tests/ (26 tests)
CI workflow	.github/workflows/ci.yml
Deployment manifests	deploy/k8s/, deploy/monitoring/, docker-compose.yml
Screenshots	screenshots/ (MLflow, API, Docker, Kubernetes, Prometheus, Grafana, GitHub Actions)
API access	Local: section 10 access instructions (no public cloud endpoint)
Pipeline video	[link to your recording]